

Deep Learning: CNNs, RNNs e Transformers

A Anatomia dos LLMs



Nexus AffillAte MMN_IA

Deep Learning: CNNs, RNNs e Transformers

A Anatomia dos Modelos que Mudaram o Mundo

Autor: Nexus Affil'IA'te MMN_IA

Coleção: Curso Universo IA — Coletânea Técnica Vol. 02

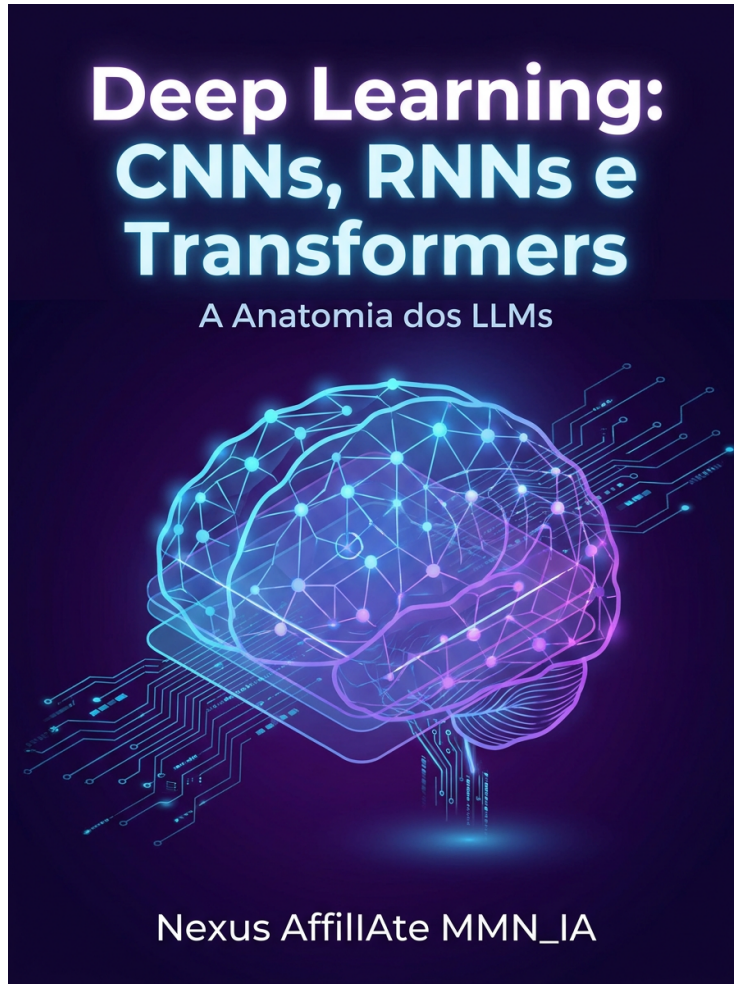
Versão: 1.0

Data: 2026-06

© 2026 Nexus Affil'IA'te MMN_IA

Licença: MIT · Distribuição livre com atribuição

title: "Deep Learning: CNNs, RNNs e Transformers"
subtitle: "A Anatomia dos Modelos que Mudaram o Mundo"
author: "Nexus Affil'IA'te MMN_IA"
collection: "Curso Universo IA — Coletânea Técnica Vol. 02"
version: "1.0"
date: "2026-06"
classification: "Acadêmico · Técnico · PhD-Level"



*"Deep Learning não é uma tecnologia. É uma mudança na forma como representamos conhecimento."**

— Nexus Affil'IA'te MMN_IA

Sumário

Prólogo — A Revolução Profunda
Capítulo 1 — O Neurônio Artificial
Capítulo 2 — A Rede Neural Feedforward
Capítulo 3 — Funções de Ativação
Capítulo 4 — Forward Propagation
Capítulo 5 — Backpropagation — A Mágica do Aprendizado
Capítulo 6 — Gradient Descent e suas Variantes
Capítulo 7 — Inicialização de Pesos
Capítulo 8 — Batch Normalization e Layer Norm
Capítulo 9 — Regularização em Deep Learning
Capítulo 10 — Otimizadores Modernos (Adam, AdamW)
Capítulo 11 — Redes Neurais Convolucionais (CNN)
Capítulo 12 — Aritmética da Convolução
Capítulo 13 — Pooling e Arquiteturas Clássicas
Capítulo 14 — ResNet, Inception, EfficientNet
Capítulo 15 — Transfer Learning e Fine-Tuning
Capítulo 16 — Redes Neurais Recorrentes (RNN)
Capítulo 17 — LSTM e GRU — Portas que Resolvem o Esquecimento
Capítulo 18 — Seq2Seq e Attention
Capítulo 19 — A Arquitetura Transformer
Capítulo 20 — Self-Attention Multi-Cabeça
Capítulo 21 — Positional Encoding
Capítulo 22 — Vision Transformers (ViT)
Capítulo 23 — Multimodal Deep Learning
Capítulo 24 — Treinamento em Escala (Distributed Training)
Capítulo 25 — O Estado da Arte em 2026
Glossário
Referências

Prólogo — A Revolução Profunda

Em 2012, três pesquisadores — **Alex Krizhevsky, Ilya Sutskever e Geoffrey Hinton** — apresentaram a **AlexNet** e venceram o ImageNet por uma margem absurda (15.3% vs. 26.2% de erro top-5). Foi o Big Bang do Deep Learning moderno. O que havia de especial? **Profundidade**. Uma rede com 8 camadas, 60 milhões de parâmetros, treinada em GPUs, que aprendeu representações hierárquicas de imagens — sozinha, sem feature engineering manual.

Esse foi o início de uma década de revolução:

- **2014:** GANs (Goodfellow), VGG, Inception.
- **2015:** ResNet (He et al.), BatchNorm, AlphaGo.
- **2016:** AlphaGo derrota Lee Sedol.
- **2017:** Transformer (Vaswani et al.) — "Attention is all you need".
- **2018:** BERT, GPT-2.
- **2020:** GPT-3 (175B parâmetros).
- **2022:** ChatGPT — IA generativa entra na cultura popular.
- **2023-2026:** Foundation models multimodais, AI agents autônomos.

Este volume é a **anatomia** desses modelos. Vamos dissecar neurônio por neurônio, camada por camada, atenção por atenção.

Capítulo 1 — O Neurônio Artificial

1.1 Inspiração Biológica

O neurônio biológico:

1. Recebe sinais nos dendritos.
2. Processa no soma.
3. Gera spike no axônio se ultrapassar threshold.

O neurônio artificial (McCulloch-Pitts, 1943; Perceptron de Rosenblatt, 1958) imita isso:

$$z = \sum_{i=1}^n w_i x_i + b, \quad a = \sigma(z)$$

1.2 O Perceptron

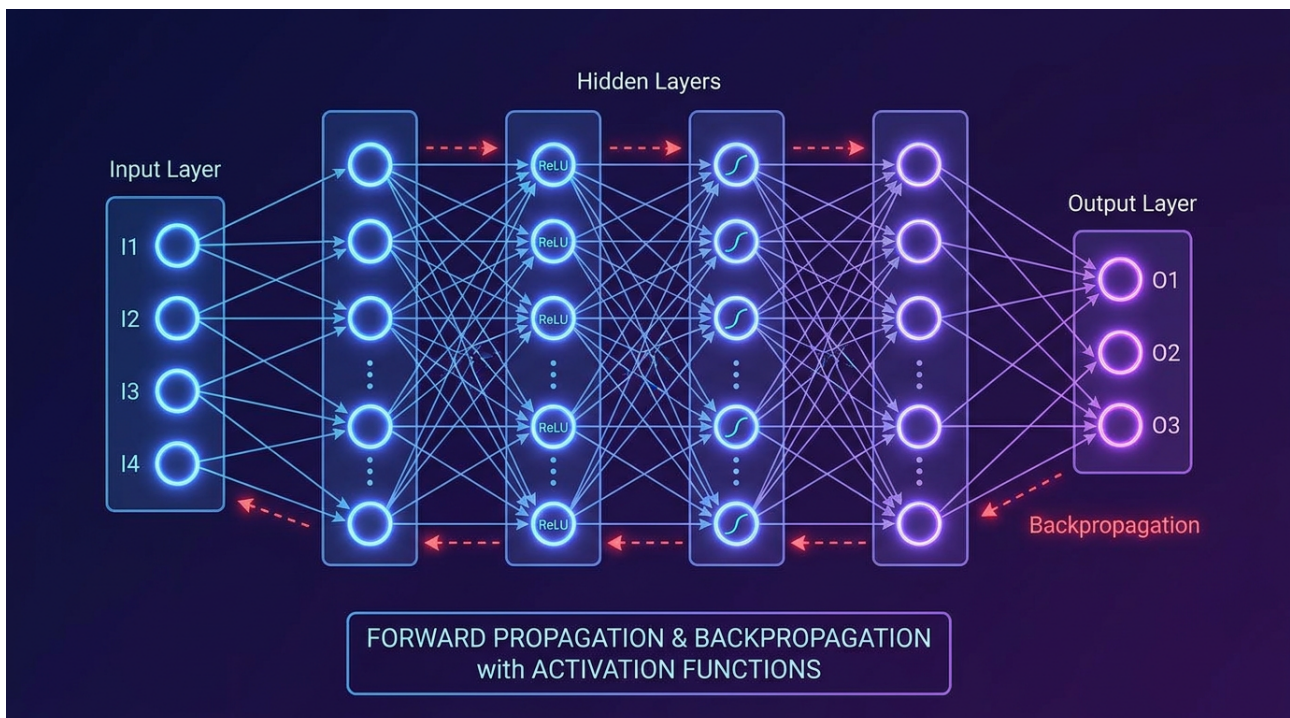
Função de ativação **step**:

$$a = \begin{cases} 1 & \text{se } z \geq 0 \\ 0 & \text{caso contrário} \end{cases}$$

Limitação: Não resolve XOR (Minsky & Papert, 1969).

1.3 Solução: Múltiplas Camadas

Empilhar perceptrons em camadas resolve problemas não-lineares. **A função de ativação não-linear** é o ingrediente mágico.



Capítulo 2 — A Rede Neural Feedforward

2.1 Arquitetura

Uma rede feedforward (MLP — Multi-Layer Perceptron) tem:

- **Camada de entrada (L_0):** n neurônios (uma por feature).
- **Camadas ocultas ($L_1, L_2, \dots, L_k$):** Processamento intermediário.
- **Camada de saída (L_{k+1}):** m neurônios (uma por classe ou 1 para regressão).

2.2 Notação

- $a^{(l)}$: Ativações da camada l
- $W^{(l)}$: Matriz de pesos da camada l (shape: $n \times n$)
- $b^{(l)}$: Vetor de bias da camada l
- $z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$
- $a^{(l)} = \sigma(z^{(l)})$

2.3 Capacidade Representacional

Teorema da Aproximação Universal (Cybenko, 1989):

Uma rede neural feedforward com uma única camada oculta e ativação sigmoideal pode aproximar **qualquer função contínua** em um compacto, com precisão arbitrária, desde que tenha neurônios suficientes.

Limitação do teorema: Não diz **quão** muitos neurônios, nem como **encontrar** os pesos certos.

2.4 Por que Profundidade > Largura?

Pesquisa recente (Eldan & Shamir, 2016; Telgarsky, 2016) mostra que redes profundas são **exponencialmente mais eficientes** que rasas para certas funções.

Capítulo 3 — Funções de Ativação

3.1 Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Problemas:

- Vanishing gradient (saturação em $\pm x$)
- Output não centrado em zero
- Custo computacional (exp)

3.2 Tanh

$$\tanh(x) = 2\sigma(2x) - 1$$

Centrada em zero, mas ainda sofre vanishing gradient.

3.3 ReLU (Rectified Linear Unit)

$$\text{ReLU}(x) = \max(0, x)$$

Vantagens:

- Computacionalmente barato
- Não satura para $x > 0$
- Induz esparsidade
- Convergência 6x mais rápida que sigmoid/tanh (Krizhevsky, 2012)

Problema: "Dying ReLU" — neurônios podem morrer (gradient = 0 para $x < 0$).

3.4 Leaky ReLU

$$\text{LeakyReLU}(x) = \max(\alpha x, x), \quad \alpha = 0.01$$

Resolve o dying ReLU.

3.5 GELU (Gaussian Error Linear Unit)

$$\text{GELU}(x) = x \cdot \Phi(x)$$

Usada em **GPT**, **BERT**, e transformers modernos.

3.6 SwiGLU e Swish

$$\text{Swish}(x) = x \cdot \sigma(\beta x)$$

$$\text{SwiGLU}(x) = \text{Swish}(xW) \otimes xV$$

Usadas em **LLaMA**, **PaLM**, **Mistral**.

3.7 Softmax (Camada de Saída)

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Converte logits em distribuição de probabilidade (classificação multi-classe).

Capítulo 4 — Forward Propagation

4.1 Algoritmo

```
def forward(x, params):
    a = x
    for l in range(L):
        z[l+1] = W[l+1] @ a + b[l+1]
        a = activation(z[l+1])
    return a
```

4.2 Vectorização

Em vez de loops, processe **batches** com matrizes:

$$Z^{[l+1]} = W^{[l+1]} A^{[l]} + b^{[l+1]}$$
 Onde A tem shape (n_features, batch_size).

4.3 Custo Computacional

Para uma camada densa com n entrada e m saída:

- **Multiplicações:** $n \cdot m$
- **Memória para ativações:** $\text{batch_size} \cdot m$

Capítulo 5 – Backpropagation – A Mágica do Aprendizado

5.1 A Ideia

Backprop (Rumelhart, Hinton, Williams, 1986) calcula o gradiente da loss em relação a **todos os pesos** da rede, de trás para frente, usando a **regra da cadeia**.

5.2 Regra da Cadeia

Se $L = f(g(h(x)))$, então:

$$\frac{dL}{dx} = \frac{df}{dg} \cdot \frac{dg}{dh} \cdot \frac{dh}{dx}$$

5.3 Algoritmo

```
def backward(y_true, y_pred, cache):
    dA[L] = - (y_true / y_pred) + ((1 - y_true) / (1 - y_pred))
    for l in reversed(range(L)):
        dZ[l] = dA[l] * activation_derivative(Z[l])
        dW[l] = (1/m) * dZ[l] @ A[l-1].T
        db[l] = (1/m) * np.sum(dZ[l], axis=1, keepdims=True)
        dA[l-1] = W[l].T @ dZ[l]
    return dW, db
```

5.4 Intuição

- **Forward:** computa a predição.
- **Backward:** mede a "culpa" de cada peso pelo erro.
- **Update:** ajusta pesos na direção que reduz erro.

5.5 Vanishing e Exploding Gradients

Em redes profundas, gradientes podem:

- **Desaparecer** $\rightarrow 0$ (ativação sigmoideal, inicialização ruim)
- **Explodir** $\rightarrow \infty$ (multiplicação por pesos grandes)

Soluções:

- Ativações ReLU
- Inicialização He/Xavier
- Batch Normalization
- Residual connections (ResNet)
- Gradient clipping

Capítulo 6 – Gradient Descent e suas Variantes

6.1 Batch Gradient Descent (BGD)

Usa **todo** o dataset para calcular o gradiente:

$$\theta := \theta - \alpha \nabla_{\theta} J(\theta)$$

Prós: Convergência suave.

Contras: Lento para datasets grandes, pode ficar preso em mínimos locais.

6.2 Stochastic Gradient Descent (SGD)

Usa **uma amostra** por vez:

$$\theta := \theta - \alpha \nabla_{\theta} J(\theta; x^{(i)}; y^{(i)})$$

Prós: Rápido, pode escapar de mínimos locais.

Contras: Ruidoso, nunca converge "exatamente".

6.3 Mini-Batch SGD

Compromisso: usa **batches** de 32-512 amostras.

- Mais eficiente computacionalmente (vetorização).
- Menos ruído que SGD puro.

6.4 SGD com Momentum

$$v_t = \beta v_{t-1} + \nabla_{\theta} J(\theta)$$

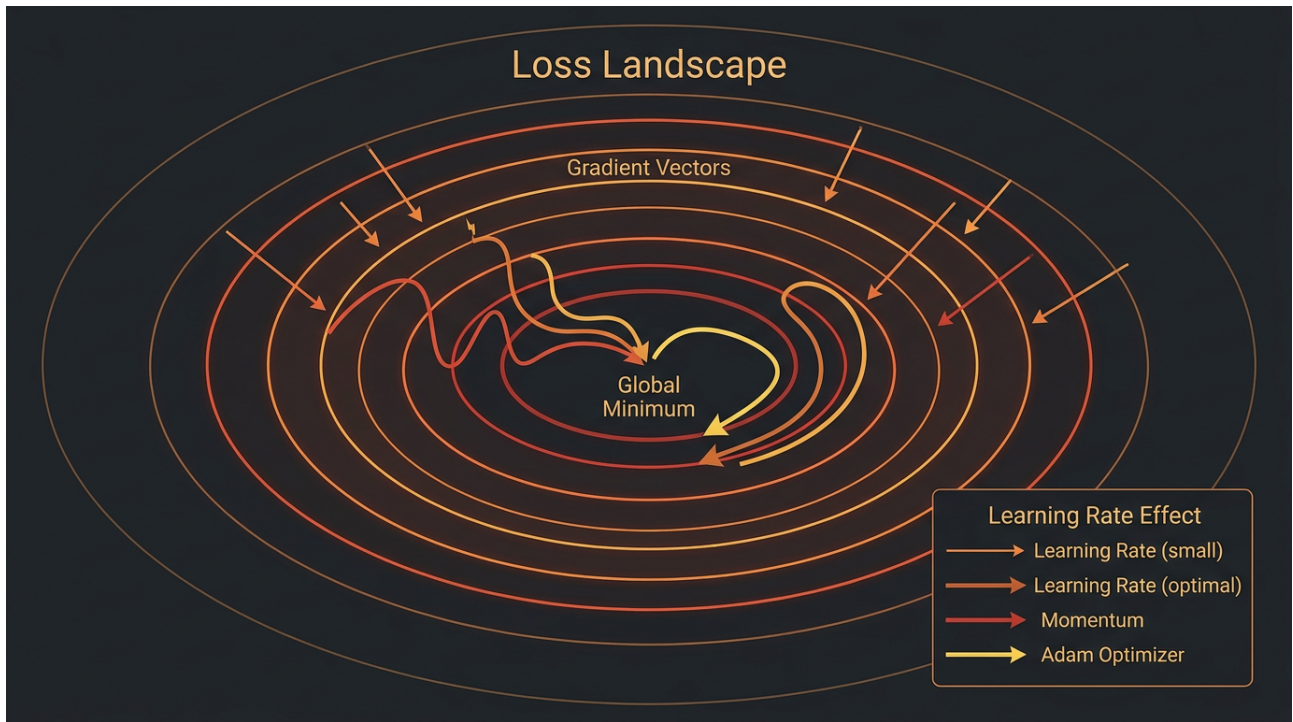
$$\theta := \theta - \alpha v_t$$

Acelera convergência em vales alongados.

6.5 Nesterov Accelerated Gradient (NAG)

Olha adiante antes de calcular o gradiente:

$$v_t = \beta v_{t-1} + \nabla_{\theta} J(\theta - \beta v_{t-1})$$



Capítulo 7 – Inicialização de Pesos

7.1 Por que Importa

Inicialização ruim = vanishing/exploding gradient. Boa inicialização = treino estável e rápido.

7.2 Xavier/Glorot (2010)

Para ativação **tanh**:

$W \sim \mathcal{N}\left(0, \frac{1}{n_{in}}\right)$ **ou** $U \sim \mathcal{N}\left(-\frac{1}{\sqrt{n_{in}}}, \frac{1}{\sqrt{n_{in}}}\right)$

7.3 He/Kaiming (2015)

Para ativação **ReLU**:

$W \sim \mathcal{N}\left(0, \frac{2}{n_{in}}\right)$

7.4 Inicialização LeCun

$W \sim \mathcal{N}\left(0, \frac{1}{n_{in}}\right)$

Para SELU e outras ativações self-normalizing.

Capítulo 8 – Batch Normalization e Layer Norm

8.1 Batch Normalization (Ioffe & Szegedy, 2015)

Normaliza as ativações de uma camada **por batch**:

$$\hat{z}^{(i)} = \frac{z^{(i)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$a^{(i)} = \gamma \hat{z}^{(i)} + \beta$$

Benefícios:

- Permite learning rates maiores
- Reduz sensibilidade à inicialização
- Atua como regularizador
- Acelera convergência

8.2 Layer Normalization

Normaliza **por feature** (cada exemplo isoladamente), não por batch. Usada em **Transformers** (ao invés de BatchNorm) porque batches em NLP têm tamanhos variáveis.

8.3 Group Normalization e Instance Norm

- **Group Norm:** Divide canais em grupos, normaliza cada grupo.
- **Instance Norm:** Normaliza por canal, útil em style transfer.

Capítulo 9 – Regularização em Deep Learning

9.1 Dropout (Srivastava et al., 2014)

Durante treino, **desativa aleatoriamente** p% dos neurônios. Força a rede a não depender de features específicas.

Inverted dropout:

```
def train_step(x):
    mask = (np.random.rand(*x.shape) < keep_prob) / keep_prob
    return x * mask
```

9.2 L1 e L2 Regularization

- **L1 (Lasso):** Adiciona λw à loss. Induz esparsidade.
- **L2 (Ridge):** Adiciona λw^2 . Penaliza pesos grandes.

9.3 Early Stopping

Monitora val loss. Quando não melhora por N epochs, para o treino e retorna ao melhor checkpoint.

9.4 Data Augmentation

Cria variações dos dados:

- **Imagens:** rotação, flip, crop, color jitter, mixup, cutmix.
- **Texto:** substituição por sinônimos, back-translation.
- **Áudio:** ruído, pitch shift, time stretch.

9.5 Label Smoothing

Substitui one-hot por $(1-\epsilon)\mathbf{y} + \epsilon/K$ para evitar overconfidence.

Capítulo 10 – Otimizadores Modernos

10.1 Adam (Kingma & Ba, 2015)

Combina momentum e adaptive learning rates:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t =$$

$$\frac{v_t}{1-\beta_2^t}$$

$$\theta := \theta - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

10.2 AdamW (Loshchilov & Hutter, 2019)

Decoupling weight decay do gradiente:

$$\theta := \theta - \alpha \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \right.$$

$$\left. \lambda \theta \right)$$

Padrão para treinar **Transformers**.

10.3 RMSProp

Adapta learning rate por parâmetro:

$$v_t = \beta v_{t-1} + (1-\beta) g_t^2$$

$$\theta := \theta - \alpha \frac{g_t}{\sqrt{v_t + \epsilon}}$$

10.4 Lion (Chen et al., 2023)

Mais memory-efficient que AdamW, com sign-based update.

10.5 Schedule Free Optimizers (2024)

Elimina a necessidade de learning rate schedules, usado em foundation models.

Capítulo 11 — Redes Neurais Convolucionais (CNN)

11.1 A Inspiração

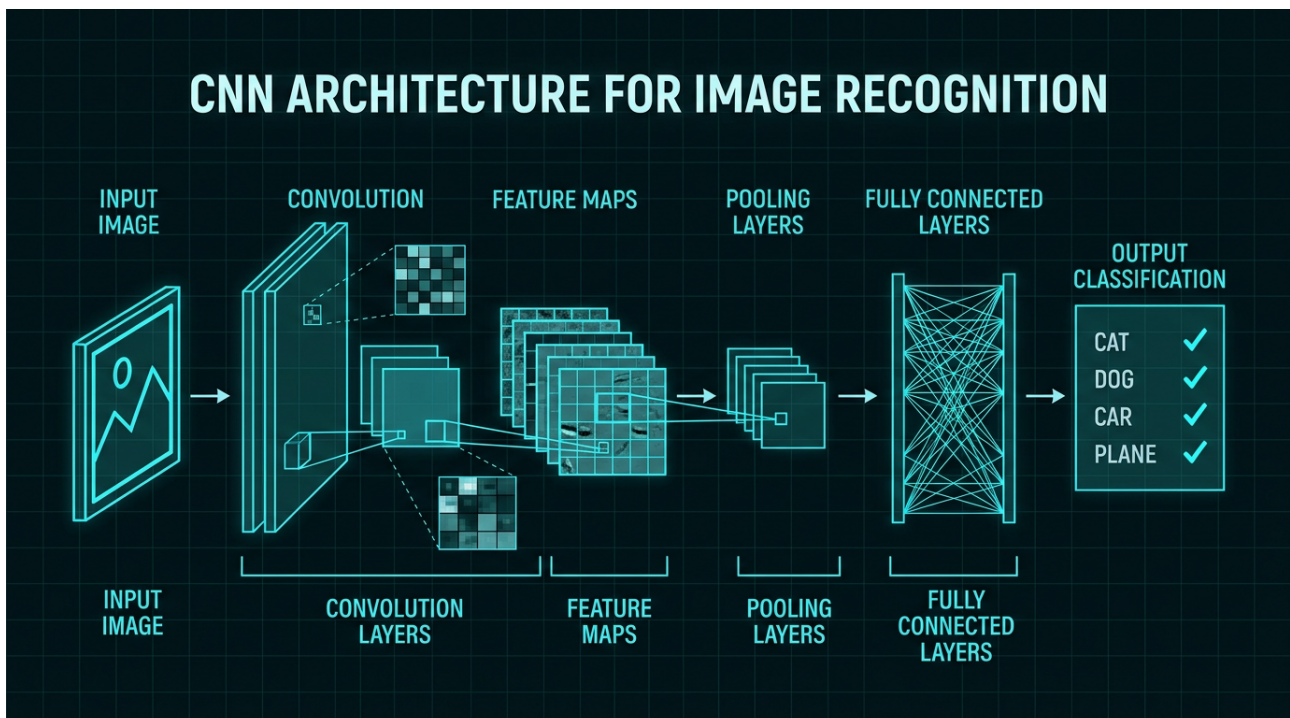
Hubel & Wiesel (1959) descobriram que o córtex visual de gatos tem neurônios que respondem a **padrões locais** (bordas, orientações). CNNs imitam isso.

11.2 Três Ideias Fundamentais

1. Conectividade local: Cada neurônio vê apenas uma região local.
2. Pesos compartilhados: O mesmo filtro é aplicado em toda a imagem.
3. Pooling: Reduz dimensionalidade e adiciona invariância.

11.3 Aplicações

- **Visão computacional:** classificação, detecção, segmentação.
- **Imagem médica:** raio-X, ressonância.
- **Self-driving cars:** detecção de pedestres, sinais.
- **Reconhecimento facial:** FaceNet, ArcFace.



Capítulo 12 — Aritmética da Convolução

12.1 Convolução Discreta 2D

$$(I * K)(i, j) = \sum_m \sum_n I(i+m, j+n) K(m, n)$$
 Onde I é a imagem e K é o kernel (filtro).

12.2 Parâmetros

- **Kernel size (k):** geralmente 3×3 ou 5×5.
- **Stride (s):** passo do filtro. s=1 mantém, s=2 reduz pela metade.
- **Padding (p):** zeros ao redor para manter dimensões.

12.3 Dimensão de Saída

$$n_{out} = \left\lfloor \frac{n_{in} + 2p - k}{s} \right\rfloor + 1$$

12.4 Tipos de Convolução

- **Standard Conv:** Cada filtro olha todos os canais de entrada.
- **Depthwise Separable Conv (MobileNet):** Fatoriza em depthwise + pointwise. 9x menos parâmetros.
- **Dilated/Atrous Conv:** Pula pixels, expande receptive field.
- **Transposed Conv:** Aumenta resolução (usada em GANs, U-Net).

Capítulo 13 – Pooling e Arquiteturas Clássicas

13.1 Max Pooling

Seleciona o maior valor em cada janela (ex: 2×2). Reduz dimensionalidade e adiciona invariância a translação.

13.2 Average Pooling

Tira a média. Menos agressivo, preserva mais informação.

13.3 Global Average Pooling

Reduz cada feature map a um único número (média de todos os pixels). Usada em vez de flatten no final de muitas CNNs.

13.4 LeNet-5 (1998)

Primeira CNN bem-sucedida, usada para dígitos MNIST.

13.5 AlexNet (2012)

8 camadas, 60M parâmetros, venceu ImageNet. Introduziu ReLU, dropout, GPU training.

13.6 VGG-16 (2014)

13 camadas convolucionais + 3 fully connected. Simples, 138M parâmetros. Demonstrou que **profundidade ajuda**.

Capítulo 14 – ResNet, Inception, EfficientNet

14.1 O Problema da Profundidade

Em 2015, redes com 50+ camadas tinham **pior** performance que redes de 20 camadas. Por quê? Otimização, não capacidade.

14.2 ResNet (He et al., 2015)

Residual Connection:

$$H(x) = F(x) + x$$

A camada aprende o **residual** $F(x) = H(x) - x$, que é mais fácil que aprender $H(x)$ do zero.

ResNet-50: 50 camadas, 25M parâmetros.

ResNet-152: 152 camadas, 60M parâmetros.

Variação: ResNeXt, Wide ResNet, DenseNet (cada camada conecta todas as anteriores).

14.3 Inception/GoogLeNet (Szegedy et al., 2014)

Aplica **múltiplos filtros** (1×1, 3×3, 5×5, pool) em paralelo, concatena resultados. Mais eficiente que uma convolução grande.

Inception-v3, v4: Refinamentos.

14.4 EfficientNet (Tan & Le, 2019)

Compound Scaling: Escala uniformemente profundidade, largura e resolução.

$$\text{depth} = \alpha^\phi, \quad \text{width} = \beta^\phi, \quad$$

$$\text{resolution} = \gamma^\phi$$

EfficientNet-B7: Melhor accuracy/FLOPs trade-off.

14.5 ConvNeXt (2022)

"Modernizar" uma CNN padrão com idéias de Transformers (depthwise conv, LayerNorm, GELU). Supera Swin Transformer em muitas tarefas.

Capítulo 15 – Transfer Learning e Fine-Tuning

15.1 O Conceito

Modelos treinados em datasets grandes (ex: ImageNet) aprendem **representações gerais**. Podemos reusar para tarefas específicas.

15.2 Estratégia

1. Carregue modelo pré-treinado (ex: ResNet-50 no ImageNet).
2. Remova a última camada (classificador).
3. Adicione nova cabeça para sua tarefa.
4. Congele as primeiras camadas (features genéricas).
5. Treine apenas a nova cabeça.
6. Fine-tune (opcional) das últimas camadas com learning rate baixo.

15.3 Por que Funciona?

Camadas iniciais aprendem features genéricas (bordas, texturas), camadas finais aprendem features específicas da tarefa. Reusar as primeiras é eficiente.

15.4 Foundation Models

Modelos treinados em escala massiva que servem como base para múltiplas tarefas:

- **CLIP (OpenAI):** Texto + Imagem.
- **DINOv2 (Meta):** Features visuais self-supervisionadas.
- **BERT/GPT:** Texto.
- **Whisper:** Áudio.

Capítulo 16 — Redes Neurais Recorrentes (RNN)

16.1 A Motivação

Redes feedforward processam cada entrada **independentemente**. Mas texto, áudio e séries temporais são **sequências** – a ordem importa.

16.2 A Arquitetura

```


$$h_t = \sigma(W_h h_{t-1} + W_x x_t + b)$$


$$y_t = \text{softmax}(W_y h_t)$$

O hidden state  $h_t$  carrega informação do passado.

```

16.3 Tipos de Sequência

- **One-to-many:** Imagem → Descrição (captioning).
- **Many-to-one:** Texto → Sentimento.
- **Many-to-many:** Tradução, vídeo captioning.
- **Sync many-to-many:** Cada timestep tem saída (tagging).

16.4 O Problema: Vanishing/Exploding Gradient

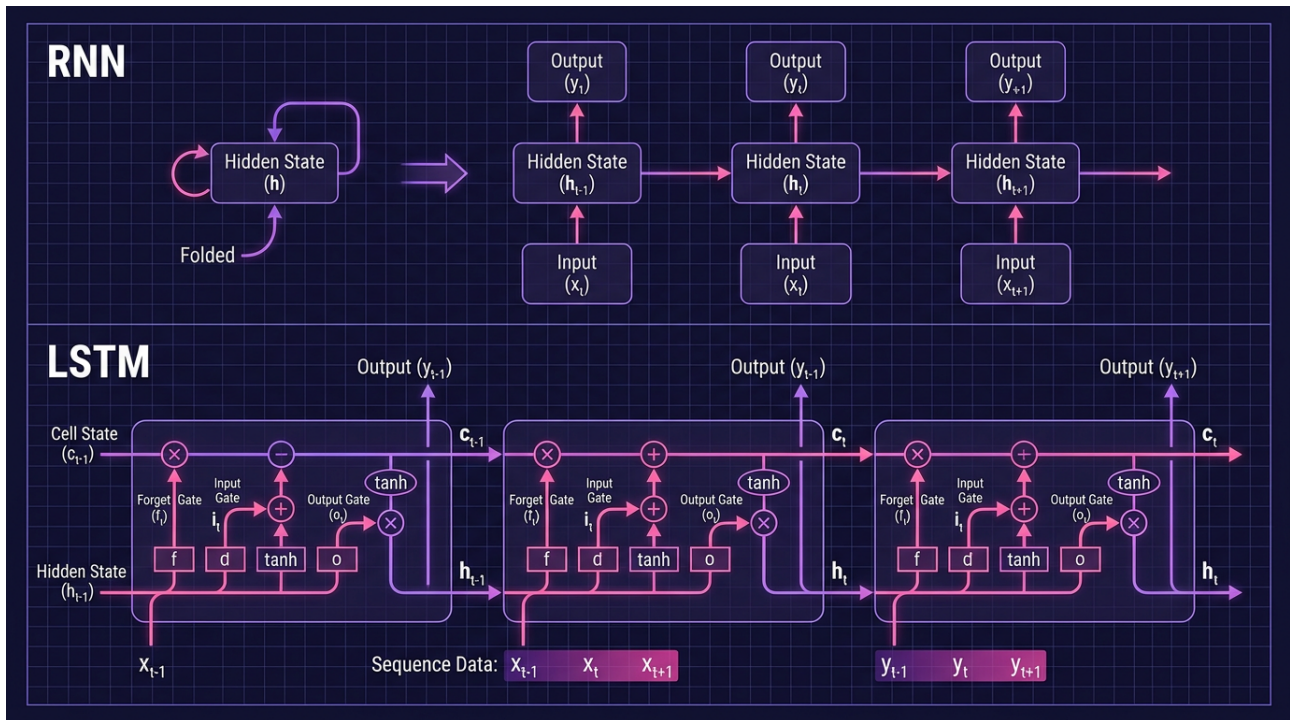
Em sequências longas, gradientes são multiplicados muitas vezes pela mesma matriz W_h :

- Se maior que 1 → explode.
- Se menor que 1 → desaparece.

Consequência: RNNs "esquecem" dependências de longo prazo.

16.5 Backpropagation Through Time (BPTT)

Aplicar backprop "desenrolando" a RNN ao longo de T timesteps. Custo computacional alto, memória alta.



Capítulo 17 – LSTM e GRU – Portas que Resolvem o Esquecimento

17.1 LSTM (Long Short-Term Memory) (Hochreiter & Schmidhuber, 1997)

LSTM resolve o vanishing gradient com **gates** que controlam o fluxo de informação:

Forget gate: $f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$

Input gate: $i_t = \sigma(W_i [h_{t-1}, x_t] + b_i)$

Candidate: $\tilde{C}_t = \tanh(W_C [h_{t-1}, x_t] + b_C)$

Cell state: $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$

Output gate: $o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$

Hidden state: $h_t = o_t \odot \tanh(C_t)$

17.2 GRU (Gated Recurrent Unit) (Cho et al., 2014)

Versão simplificada do LSTM, com 2 gates ao invés de 3:

- **Update gate:** Decide quanto do passado manter.
- **Reset gate:** Decide quanto do passado esquecer.

17.3 Performance

- **LSTM** $\approx$ **GRU** em performance, GRU é mais rápido.
- Ambos superam vanilla RNN em sequências longas.
- Transformers superam ambos em NLP moderno.

Capítulo 18 – Seq2Seq e Attention

18.1 Seq2Seq (Sutskever et al., 2014)

Encoder-Decoder para tradução:

- **Encoder:** Lê sequência inteira, gera contexto c .
- **Decoder:** Gera saída a partir de c .

Problema: c tem tamanho fixo. Sequências longas comprimem informação.

18.2 Attention Mechanism (Bahdanau et al., 2015)

Em vez de usar um único c , **attention** permite ao decoder olhar para **todos os hidden states** do encoder, pesando por relevância:

$$\alpha_t = \text{softmax}(e_t), \quad e_{t,i} = \text{score}(s_{t-1}, h_i)$$

$$c_t = \sum_i \alpha_{t,i} h_i$$

18.3 Self-Attention

Attention **dentro da mesma sequência:**

- Cada posição atende a todas as posições.
- Captura dependências de longo alcance.
- **Base dos Transformers.**

Capítulo 19 — A Arquitetura Transformer

19.1 O Paper

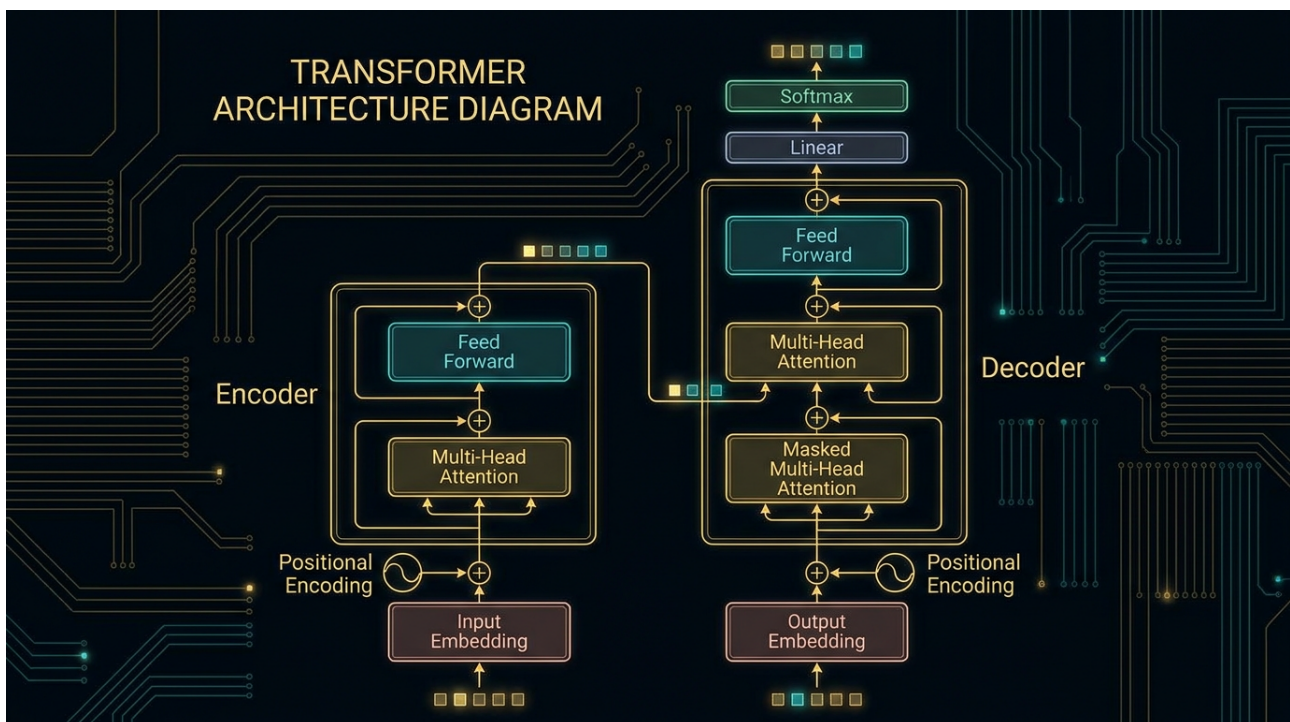
"Attention is all you need" (Vaswani et al., 2017) eliminou RNNs e CNNs em favor de **apenas attention**.

19.2 Arquitetura Geral

```
Input → Embedding + Positional Encoding
→ N × [Multi-Head Self-Attention + Add&Norm + FFN + Add&Norm]
→ Output
```

Encoder: 6 camadas idênticas.

Decoder: 6 camadas, com masked self-attention + encoder-decoder attention.



19.3 Componentes

1. Embedding – Mapeia tokens em vetores densos.
2. Positional Encoding – Adiciona informação de posição.
3. Multi-Head Attention – Múltiplas atenções em paralelo.
4. Feed-Forward Network (FFN) – 2 camadas densas com ReLU/GELU.
5. Add & Norm – Conexão residual + LayerNorm.

19.4 Vantagens sobre RNN

- **Paralelização:** Atenção é $O(1)$ em distância, RNN é $O(n)$.
- **Long-range dependencies:** Conexão direta entre quaisquer posições.
- **Treinamento mais rápido** em hardware moderno.

19.5 Complexidade

Por camada:

- **Self-attention:** $O(n^2 \cdot d)$ – atenção quadrática no comprimento.
- **FFN:** $O(n \cdot d^2)$

A complexidade quadrática no comprimento é o **gargalo** – alvo de muitas otimizações (Longformer, Flash Attention).

Capítulo 20 — Self-Attention Multi-Cabeça

20.1 Scaled Dot-Product Attention

$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$

Interpretação:

- Q (Query): O que estou procurando?
- K (Key): O que cada posição oferece?
- V (Value): Informação a ser agregada.
- QK^T : Similaridade entre queries e keys.
- $\sqrt{d_k}$: Scaling para evitar softmax saturado.

20.2 Multi-Head

Em vez de uma atenção, fazemos h atenções em paralelo, cada uma com diferentes projeções:

$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$

$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

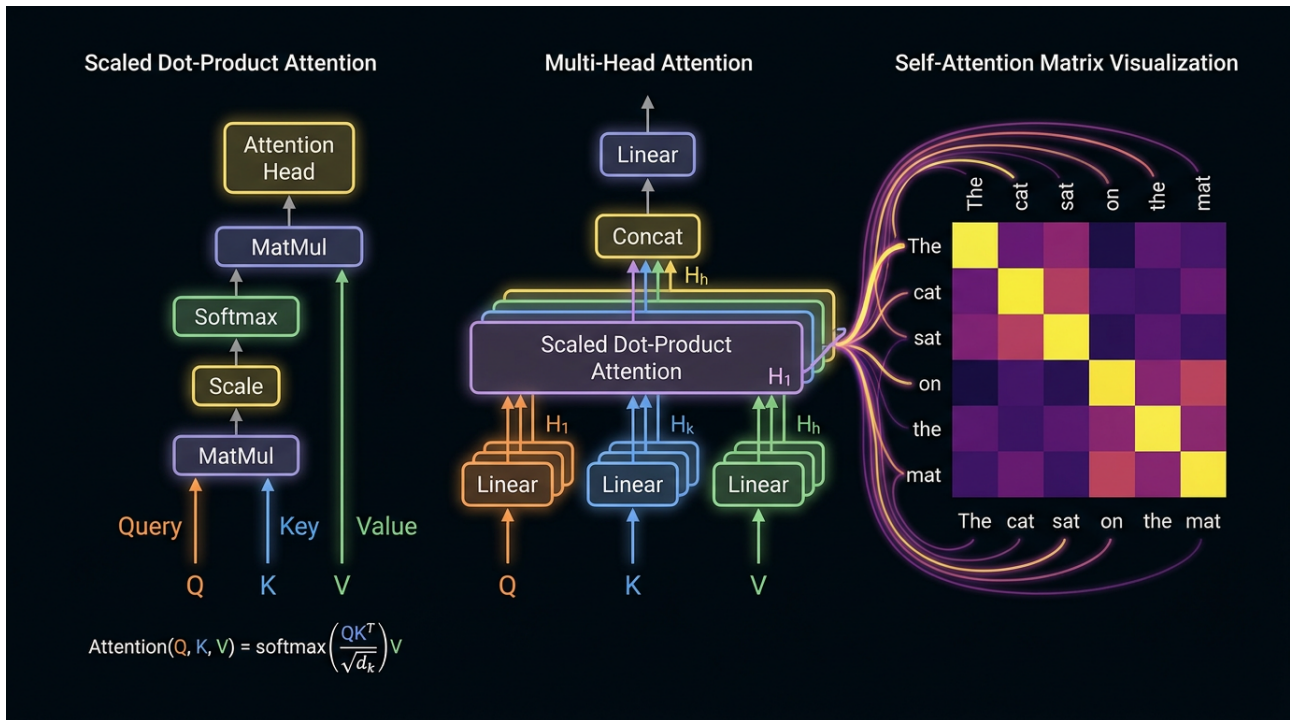
Por que múltiplas cabeças? Cada cabeça aprende a focar em diferentes aspectos (sintaxe, semântica, posição).

20.3 Attention Masking

- **Padding mask:** Ignora tokens de padding.
- **Causal mask:** No decoder, impede olhar para o futuro.

20.4 Attention Visualizations

Pesos de atenção revelam o que o modelo "olha" —útil para interpretabilidade.



Capítulo 21 – Positional Encoding

21.1 O Problema

Self-attention é **permutation-equivariant**: trocar tokens de ordem não muda o output. Precisamos injetar informação de posição.

21.2 Positional Encoding Sinusoidal (Transformer original)

$$PE_{(pos, 2i)} = \sin(pos / 10000^{2i/d})$$

$$PE_{(pos, 2i+1)} = \cos(pos / 10000^{2i/d})$$

Vantagens:

- Cada posição tem encoding único.
- Pode generalizar para sequências mais longas.
- Suave e contínuo.

21.3 Learned Positional Embeddings

Em BERT e GPT, posições são aprendidas como embeddings normais.

21.4 Rotary Position Embedding (RoPE)

Usado em **LLaMA**, **Mistral**, **PaLM**. Codifica posição com rotação no espaço:

$$q'_i = R_{\{\theta, i\}} q_i, \quad k'_i = R_{\{\theta, i\}} k_i$$

Vantagem: Generaliza bem para comprimentos não vistos no treino.

21.5 ALiBi (Attention with Linear Biases)

Adiciona bias linear à attention matrix. Mais simples, boa generalização.

Capítulo 22 – Vision Transformers (ViT)

22.1 A Ideia (Dosovitskiy et al., 2020)

Aplicar Transformer puro em imagens:

1. Divide a imagem em patches (ex: 16×16).
2. Achate cada patch em um vetor.
3. Linear project para um embedding.
4. Adicione positional embeddings.
5. Passe por Transformer encoder padrão.

22.2 ViT-Base/Large/Huge

- **ViT-B:** 86M parâmetros, 12 camadas.
- **ViT-L:** 307M parâmetros, 24 camadas.
- **ViT-H:** 632M parâmetros, 32 camadas.

22.3 ViT vs. CNN

Aspecto	CNN	ViT
Inductive bias	Localidade, translation equivariance	Fracas (precisa de muito dado)
Data efficiency	Bom em datasets pequenos	Precisa de milhões de imagens
Scaling	Saturates	Improves com scale
Interpretabilidade	Filtros visíveis	Attention maps

22.4 Híbridos CNN-Transformer

- **CoAtNet:** Convolução nos estágios iniciais, attention nos finais.
- **Swin Transformer:** Hierarchical + shifted windows.
- **ConvNeXt:** CNN com idéias de Transformer.

Capítulo 23 — Multimodal Deep Learning

23.1 Os 4 Paradigmas

1. Late Fusion: Modelos separados, combine features no final.
2. Early Fusion: Combine no nível de input.
3. Cross-Attention: Attention entre modalidades.
4. Unified Tokenization: Mesma representação para todas modalidades.

23.2 CLIP (Contrastive Language-Image Pre-training)

Treina um modelo com dois encoders (texto + imagem) usando **contrastive loss**

$$L = -\frac{1}{N} \sum_i \log \frac{\exp(\text{sim}(I_i, T_i))}{\sum_j \exp(\text{sim}(I_i, T_j))}$$

23.3 Stable Diffusion / DALL-E

Modelos de **diffusion** que geram imagens a partir de texto.

23.4 Gemini (Google DeepMind)

Modelo **nativamente multimodal** — texto, imagem, áudio, vídeo em um único modelo desde o pré-treinamento.

Capítulo 24 – Treinamento em Escala

24.1 Os Desafios

- **Memória:** Modelos grandes não cabem em uma GPU.
- **Tempo:** Treinar GPT-4 custou ~3 meses em 25k GPUs.
- **Comunicação:** Sincronizar gradientes em milhares de GPUs.

24.2 Data Parallelism

Cada GPU tem uma cópia do modelo, processa um mini-batch diferente. Sincroniza gradientes via **all-reduce**.

24.3 Model Parallelism

Divide o modelo entre GPUs:

- **Pipeline parallelism:** Camadas diferentes em GPUs diferentes.
- **Tensor parallelism:** Operações matriciais divididas.
- **Expert parallelism (MoE):** Cada token ativa apenas alguns "experts".

24.4 ZeRO (Zero Redundancy Optimizer)

Da DeepSpeed (Microsoft). Divide optimizer state, gradients e parameters entre GPUs.

24.5 Mixed Precision Training

Usar **FP16/BF16** para forward/backward, **FP32** para atualização de pesos. Reduz memória e acelera treino.

24.6 Flash Attention (Dao et al., 2022)

Reformula attention para ser **IO-aware** – reduz memória de $O(n^2)$ para $O(n)$ e acelera 2-4x.

24.7 FSDP (Fully Sharded Data Parallel)

Equivalente do ZeRO-3 no PyTorch nativo.

Capítulo 25 – O Estado da Arte em 2026

25.1 Foundation Models Dominantes

Modelo	Empresa	Parâmetros	Modalidades
GPT-4o / GPT-5	OpenAI	~trilhões (MoE)	Texto, visão, áudio
Claude 4 Opus	Anthropic	?	Texto, visão
Gemini 2.0 Ultra	Google DeepMind	trilhões	Texto, visão, áudio, vídeo
Llama 4	Meta	70B-405B	Texto, visão
Mistral Large 3	Mistral	123B	Texto
Grok 3	xAI	?	Texto, visão
DeepSeek V4	DeepSeek	670B (MoE)	Texto, código

25.2 Tendências

- Mixture of Experts (MoE):** Ativação esparsa, escala de parâmetros sem escala de compute.
- Long Context:** 1M+ tokens (Gemini 1.5 Pro, Claude 3.5).
- Reasoning Models:** o1, o3, R1 – chain-of-thought latente.
- Agentic AI:** LLMs que usam ferramentas e tomam ações.
- Synthetic Data:** Modelos treinando modelos.
- Constitutional AI:** Alinhamento por princípios escritos.

25.3 O Que Falta Resolver

- Hallucination:** Modelos ainda inventam fatos.
- Reasoning:** LLM ainda falha em lógica multi-step complexa.
- Eficiência:** Treinar um modelo grande custa milhões de dólares.
- Alignment:** Como garantir que IAs façam o que queremos, especialmente em agentic settings.
- Interpretabilidade:** Mechanistic interpretability (Anthropic) está apenas começando.

Glossário

Termo	Definição
Backpropagation	Algoritmo para calcular gradientes em redes neurais.
CNN	Rede Neural Convolucional.
Embedding	Representação densa de dados discretos em .
Fine-Tuning	Ajuste de modelo pré-treinado em tarefa específica.
Gradient Descent	Algoritmo de otimização.
Layer Norm	Normalização por feature.
LSTM	Long Short-Term Memory.
Momentum	Técnica para acelerar gradient descent.
ReLU	Função de ativação linear retificada.
RNN	Rede Neural Recorrente.
Self-Attention	Attention dentro da mesma sequência.
Token	Unidade básica processada por um modelo.
Transformer	Arquitetura baseada em atenção.
Vanishing Gradient	Gradiente que se torna muito pequeno.

Referências

1. LeCun, Y., Bengio, Y., Hinton, G. (2015). Deep learning. Nature.
2. Goodfellow, I., Bengio, Y., Courville, A. (2016). Deep Learning. MIT Press.
3. He, K. et al. (2016). Deep Residual Learning for Image Recognition. CVPR.
4. Vaswani, A. et al. (2017). Attention is all you need. NeurIPS.
5. Hochreiter, S., Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation.
6. Devlin, J. et al. (2018). BERT: Pre-training of Deep Bidirectional Transformers. NAACL.
7. Brown, T. et al. (2020). Language Models are Few-Shot Learners. NeurIPS.
8. Dosovitskiy, A. et al. (2020). An Image is Worth 16x16 Words. ICLR.
9. Kingma, D., Ba, J. (2015). Adam: A Method for Stochastic Optimization. ICLR.
10. Ioffe, S., Szegedy, C. (2015). Batch Normalization. ICML.
11. Srivastava, N. et al. (2014). Dropout. JMLR.
12. Dao, T. et al. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention. NeurIPS.
13. Bahdanau, D. et al. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. ICLR.
14. Mnih, V. et al. (2015). Human-level control through deep reinforcement learning. Nature.

Fim do Volume 02

"O Transformer é a invenção mais importante em Deep Learning desde o backpropagation. Ele mudou não só o que a IA pode fazer, mas o que podemos imaginar."

– Nexus Affil'IA'te MMN_IA



O futuro pertence a quem
ensina as máquinas a aprender.

Nexus AffillAte MMN_IA

© 2026 Nexus Affil'IA'te MMN_IA · Curso Universo IA · Coletânea Técnica